

Topic 1: Merge Sort

Question 1 (Easy) – Rank Students by Marks

A school has recently conducted a scholarship examination for all students of Class 12. The examination department has collected the marks of all students in an array. Due to the large number of participants, manually arranging the marks in ascending order has become difficult.

Your task is to write a program that takes the marks of N students and sorts them in ascending order using the Merge Sort algorithm.

Merge Sort works by repeatedly dividing the array into smaller subarrays until each subarray contains a single element. These smaller subarrays are then merged in a sorted manner to produce the final sorted array.

The school administration wants the marks to be displayed in increasing order so they can easily identify the lowest and highest scoring students and generate performance reports.

Input

- First line contains an integer N , representing the number of students.
- Second line contains N space-separated integers, representing the marks of the students.

Output

- Print the marks in ascending order.

Constraints

- $1 \leq N \leq 10^5$
- $0 \leq \text{Marks} \leq 100$

Example

Input:

```
6
78 45 92 34 67 81
```

Output:

```
34 45 67 78 81 92
```

Date -

Date

Page No.

Kanishk

Week - 3 : Module

Topic - 1 : Merge Sort -

Merge sort in C is a highly efficient, stable, and recursive algorithm that follows the divide - and - conquer strategy. It works by repeatedly dividing an array into two halves, sorting each half, and then merging the sorted halves back together.

* Full C Implementation -

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void merge (int arr[], int l, int m, int r) {
```

```
    int n1 = m - l + 1, n2 = r - m;
```

```
    int *L = (int *) malloc (n1 * size of (int)),
```

```
        *R = (int *) malloc (n2 * size of (int));
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
```

```
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
```

```
    int i = 0, j = 0, k = l;
```

```
    while (i < n1 && j < n2) arr[k++] = (L[i] < R[j]) ?
```

```
        L[i++] : R[j++];
```

```
    while (i < n1) arr[k++] = L[i++];
```

```
    while (j < n2) arr[k++] = R[j++];
```

```
    free(L); free(R);
```

```
}
```

```
void mergesort (int arr[], int l, int r) {
```

```
    if (l < r) {
```

```
        int m = l + (r - l) / 2;
```

```
        mergesort (arr, l, m);
```

```
        mergesort (arr, m + 1, r);
```

```
merge Carr, l, m, r);  
}  
}
```

```
int main() {  
    int arr[] = {38, 27, 43, 3, 9, 82, 10};  
    int n = sizeof(arr) / sizeof(arr[0]);  
    mergesort Carr, 0, n-1);  
    for (int i=0; i<n; i++) printf("%d", arr[i]);  
    return 0;  
}
```

- Time Complexity: $O(n \log n)$ (Consistent best/worst case).
- Space Complexity: $O(n)$ auxiliary space.
- Stability: Stable sorting algorithm

This algorithm is ideal for large datasets & linked lists due to its consistent performance.

Topic-1 : Solution-1 →

```
#include <stdio.h>
#include <stdlib.h>
void merge (int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int *left_sub = (int *) malloc (n1 * sizeof (int));
    int *right_sub = (int *) malloc (n2 * sizeof (int));
    for (i = 0; i < n1; i++)
        left_sub[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        right_sub[j] = arr[mid + 1 + j];
    i = 0;
    j = 0;
    k = left;
    while (i < n1 && j < n2) {
        if (left_sub[i] <= right_sub[j]) {
            arr[k] = left_sub[i];
            i++;
        } else {
            arr[k] = right_sub[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k] = left_sub[i];
        i++;
        k++;
    }
}
```

free (left_sub);

free (right_sub);

}

void mergesort (int arr[], int left, int right) {

if (left < right) {

int mid = left + (right - left) / 2;

mergesort (arr, left, mid);

mergesort (arr, mid+1, right);

mergesort (arr, left, mid, right);

}

}

int main() {

int n;

if (scanf ("%d", &n) != 1) return 0;

int *marks = (int *) malloc (n * sizeof(int));

if (marks == Null) {

return 1;

}

for (int i = 0; i < n; i++) {

if (scanf ("%d", &marks[i]) != 1) return 1;

}

mergesort (marks, 0, n-1);

for (int i = 0; i < n; i++) {

printf ("%d", marks[i]);

if (i < n-1) {

printf (" ");

}

}

printf ("\n");

free (marks);

return 0;

}



main.c

```
1  #include <stdio.h>
2
3  // Function to merge two halves of an array
4  void merge(int arr[], int left, int mid, int right) {
5      int n1 = mid - left + 1;
6      int n2 = right - mid;
7
8      // Temporary arrays
9      int L[n1], R[n2];
10
11     // Copy data to temporary arrays L[] and R[]
12     for (int i = 0; i < n1; i++)
13         L[i] = arr[left + i];
14     for (int j = 0; j < n2; j++)
15         R[j] = arr[mid + 1 + j];
16
17     int i = 0, j = 0, k = left;
18
19     // Merge the temporary arrays back into arr[left..right]
20     while (i < n1 && j < n2) {
21         if (L[i] <= R[j]) {
22             arr[k] = L[i];
23             i++;
24         } else {
25             arr[k] = R[j];
26             j++;
27         }
28         k++;
29     }
30
31     // Copy the remaining elements of L[], if there are any
32     while (i < n1) {
33         arr[k] = L[i];
34         i++;
35         k++;
36     }
```


main.c

```
27     }
28     k++;
29 }
30
31 // Copy the remaining elements of L[], if there are any
32 while (i < n1) {
33     arr[k] = L[i];
34     i++;
35     k++;
36 }
37
38 // Copy the remaining elements of R[], if there are any
39 while (j < n2) {
40     arr[k] = R[j];
41     j++;
42     k++;
43 }
44 }
45
46 // Main function to perform Merge Sort
47 void mergeSort(int arr[], int left, int right) {
48     if (left < right) {
49         // Calculate mid point to avoid overflow
50         int mid = left + (right - left) / 2;
51
52         // Sort first and second halves
53         mergeSort(arr, left, mid);
54         mergeSort(arr, mid + 1, right);
55
56         // Merge the sorted halves
57         merge(arr, left, mid, right);
58     }
59 }
60
```

main.c

```
43     }
44 }
45
46 // Main function to perform Merge Sort
47 void mergeSort(int arr[], int left, int right) {
48     if (left < right) {
49         // Calculate mid point to avoid overflow
50         int mid = left + (right - left) / 2;
51
52         // Sort first and second halves
53         mergeSort(arr, left, mid);
54         mergeSort(arr, mid + 1, right);
55
56         // Merge the sorted halves
57         merge(arr, left, mid, right);
58     }
59 }
60
61 int main() {
62     int N;
63     scanf("%d", &N); // Number of students
64
65     int marks[N];
66     for(int i = 0; i < N; i++) {
67         scanf("%d", &marks[i]);
68     }
69
70     mergeSort(marks, 0, N - 1);
71
72     for(int i = 0; i < N; i++) {
73         printf("%d ", marks[i]);
74     }
75     printf("\n");
76
77     return 0;
78 }
```



```

52 // Sort first and second halves
53 mergeSort(arr, left, mid);
54 mergeSort(arr, mid + 1, right);
55
56 // Merge the sorted halves
57 merge(arr, left, mid, right);
58 }
59 }
60
61 int main() {
62     int N;
63     scanf("%d", &N); // Number of students
64
65     int marks[N];
66     for(int i = 0; i < N; i++) {
67         scanf("%d", &marks[i]);
68     }
69
70     mergeSort(marks, 0, N - 1);

```



6 78 45 92 34 67 81

34 45 67 78 81 92

...Program finished with exit code 0

Press ENTER to exit console.

Solution-2:

```
#include <stdio.h>
#include <stdlib.h>
void merge (int arr [], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int *L = (int*) malloc (n1 * sizeof (int));
    int *R = (int*) malloc (n2 * sizeof (int));
    for (i = 0; i < n1; i++)
        L[i] = arr [left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr [mid + 1 + j];
    i = 0;
    j = 0;
    k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
}
```

```

while (j < n) {
    arr[L] = R[j];
    j++;
    R++;
}

```

```

}
free(L);
free(R);
}

```

```

void mergesort (int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergesort (arr, left, mid);
        mergesort (arr, mid + 1, right);
        merge (arr, left, mid, right);
    }
}

```

```

}
int main() {
    int n;
    if (scanf ("%d", &n) != 1) return 0;
    int * prices = (int *) malloc (n * sizeof(int));
    if (prices == Null) {
        return 1;
    }
}

```

```

for (int i = 0; i < n; i++) {
    if (scanf ("%d", &prices[i]) != 1) return 0;
}

```

```

mergesort (prices, 0, n - 1);
for (int i = 0; i < n; i++) {
    printf ("%d", prices[i]);
    if (i < n - 1) {
        printf (" ");
    }
}
}

```



```
printf("\n");  
free(prices);  
return 0;  
}
```

Solution-3 :-

```
#include <stdio.h>  
#include <stdlib.h>  
void merge (int arr[], int left, int mid, int right) {  
    int i, j, k;  
    int n1 = mid - left + 1;  
    int n2 = right - mid;  
    int *L = (int *) malloc (n1 * sizeof (int));  
    int *R = (int *) malloc (n2 * sizeof (int));  
    for (i = 0; i < n1; i++)  
        L[i] = arr[left + i];  
    for (j = 0; j < n2; j++)  
        R[j] = arr[mid + 1 + j];  
    i = 0;  
    j = 0;  
    k = left;  
    while (i < n1 && j < n2) {  
        if (L[i] <= R[j]) {  
            arr[k] = L[i];  
            i++;  
        } else {  
            arr[k] = R[j];  
            j++;  
        }  
        k++;  
    }  
}
```

```

while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

```

```

while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}

```

```

free(L);

```

```

free(R);

```

```

}

```

```

void mergesort (int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + (right - left) / 2);
        mergesort (arr, left, mid);
        mergesort (arr, mid + 1, right);
        merge (arr, left, mid, right);
    }
}

```

```

}

```

```

int main() {

```

```

    int n;

```

```

    if (scanf("%d", &n) != 1) return 1;

```

```

    int *salaries = (int *) malloc (n * sizeof(int));

```

```

    if (salaries == null) return 1;

```

```

    for (int i = 0; i < n; i++) {

```

```

        if (scanf("%d", &salaries[i]) != 1) return 1;
    }

```

```

    mergesort (salaries, 0, n-1);

```

```

    for (int i = 0; i < n; i++) {

```



```
printf("%d", salaries[i]);  
if (i < n - 1) {  
    printf(" ");  
}  
}  
printf("\n");  
int lowest_salary = salaries[0];  
int highest_salary = salaries[n - 1];  
int difference = highest_salary - lowest_salary;  
printf("Lowest salary: %d\n", lowest_salary);  
printf("Highest salary: %d\n", highest_salary);  
printf("Difference: %d\n", difference);  
free(salaries);  
return 0;  
}
```




main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void merge(long long arr[], int left, int mid, int right) {
5     int n1 = mid - left + 1;
6     int n2 = right - mid;
7     long long L[n1], R[n2];
8
9     for (int i = 0; i < n1; i++) L[i] = arr[left + i];
10    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
11
12    int i = 0, j = 0, k = left;
13    while (i < n1 && j < n2) {
14        if (L[i] <= R[j]) {
15            arr[k++] = L[i++];
16        } else {
17            arr[k++] = R[j++];
18        }
19    }
20    while (i < n1) arr[k++] = L[i++];
21    while (j < n2) arr[k++] = R[j++];
22 }
23
24 void mergeSort(long long arr[], int left, int right) {
25     if (left < right) {
26         int mid = left + (right - left) / 2;
27         mergeSort(arr, left, mid);
28         mergeSort(arr, mid + 1, right);
29         merge(arr, left, mid, right);
30     }
31 }
32
33 int main() {
34     int N;
35     scanf("%d", &N);
36 }
```

```

19     }
20     while (i < n1) arr[k++] = L[i++];
21     while (j < n2) arr[k++] = R[j++];
22 }
23
24 void mergeSort(long long arr[], int left, int right) {
25     if (left < right) {
26         int mid = left + (right - left) / 2;
27         mergeSort(arr, left, mid);
28         mergeSort(arr, mid + 1, right);
29         merge(arr, left, mid, right);
30     }
31 }
32
33 int main() {
34     int N;
35     scanf("%d", &N);
36
37     // Using dynamically allocated array for large constraints
38     long long *prices = (long long *)malloc(N * sizeof(long long));
39
40     for(int i = 0; i < N; i++) {
41         scanf("%lld", &prices[i]);
42     }
43
44     mergeSort(prices, 0, N - 1);
45
46     for(int i = 0; i < N; i++) {
47         printf("%lld ", prices[i]);
48     }
49     printf("\n");
50
51     free(prices);
52     return 0;
53 }

```

```

30     }
31 }
32
33 int main() {
34     int N;
35     scanf("%d", &N);
36
37     // Using dynamically allocated array for large constraints
38     long long *prices = (long long *)malloc(N * sizeof(long long));
39
40     for(int i = 0; i < N; i++) {
41         scanf("%lld", &prices[i]);
42     }
43
44     mergeSort(prices, 0, N - 1);
45

```



```

5 2500 1500 4000 3000 1000
1000 1500 2500 3000 4000

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```


main.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void merge(int arr[], int left, int mid, int right) {
5      int n1 = mid - left + 1, n2 = right - mid;
6      int L[n1], R[n2];
7
8      for (int i = 0; i < n1; i++) L[i] = arr[left + i];
9      for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
10
11     int i = 0, j = 0, k = left;
12     while (i < n1 && j < n2) {
13         if (L[i] <= R[j]) arr[k++] = L[i++];
14         else arr[k++] = R[j++];
15     }
16     while (i < n1) arr[k++] = L[i++];
17     while (j < n2) arr[k++] = R[j++];
18 }
19
20 void mergeSort(int arr[], int left, int right) {
21     if (left < right) {
22         int mid = left + (right - left) / 2;
23         mergeSort(arr, left, mid);
24         mergeSort(arr, mid + 1, right);
25         merge(arr, left, mid, right);
26     }
27 }
28
29 int main() {
30     int N;
31     scanf("%d", &N);
32
33     int *salaries = (int *)malloc(N * sizeof(int));
34     for(int i = 0; i < N; i++) {
35         scanf("%d", &salaries[i]);
36     }
```

```

18 }
19
20 void mergeSort(int arr[], int left, int right) {
21     if (left < right) {
22         int mid = left + (right - left) / 2;
23         mergeSort(arr, left, mid);
24         mergeSort(arr, mid + 1, right);
25         merge(arr, left, mid, right);
26     }
27 }
28
29 int main() {
30     int N;
31     scanf("%d", &N);
32
33     int *salaries = (int *)malloc(N * sizeof(int));
34     for(int i = 0; i < N; i++) {
35         scanf("%d", &salaries[i]);
36     }
37
38     mergeSort(salaries, 0, N - 1);
39
40     // 1. Sorted Salaries
41     for(int i = 0; i < N; i++) {
42         printf("%d ", salaries[i]);
43     }
44     printf("\n");
45
46     // 2. Lowest, Highest and Difference
47     printf("Lowest Salary: %d\n", salaries[0]);
48     printf("Highest Salary: %d\n", salaries[N - 1]);
49     printf("Difference: %d\n", salaries[N - 1] - salaries[0]);
50
51     free(salaries);
52     return 0;
53 }

```

```

27 }
28
29 int main() {
30     int N;
31     scanf("%d", &N);
32
33     int *salaries = (int *)malloc(N * sizeof(int));
34     for(int i = 0; i < N; i++) {
35         scanf("%d", &salaries[i]);
36     }
37
38     mergeSort(salaries, 0, N - 1);
39
40     // 1. Sorted Salaries
41     for(int i = 0; i < N; i++) {
42         printf("%d ", salaries[i]);
43     }
44     printf("\n");
45

```



25000 30000 45000 50000 60000 70000 90000

Lowest Salary: 25000

Highest Salary: 90000

Difference: 65000

...Program finished with exit code 0

Press ENTER to exit console.

Topic-2 : Quick Sort

Quick Sort is a highly efficient, in-place sorting algorithm that uses a divide and conquer strategy. It works by selecting a pivot element, partitioning the surrounding elements so that smaller values move to the left and larger values move to the right, and then recursively sorting the resulting subarrays.

Implementation in C —

```
#include <stdio.h>

void swap(int *a, int *b) { int t = *a ; *a = *b ;
                             *b = t ; }

int Partition(int arr[], int low, int high) {
    int pivot = arr[high], i = (low - 1);
    for (int j = low ; j < high ; j++) {
        if (arr[j] <= pivot) swap(&arr[++i], &arr[j]);
    }
    swap(&arr[i+1], &arr[high]);
    return (i+1);
}

void quicksort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quicksort(arr, low, pi-1);
        quicksort(arr, pi+1, high);
    }
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
```

```
quicksort(arr, 0, n-1);  
for (int i=0; i<n; i++) printf("%d", arr[i]);  
return 0;  
}
```

- Time Complexity :- ~~$O(n \log n)$~~
 $O(n \log n)$ (Average / Best), $O(n^2)$ (Worst)

- Space :-
 $O(\log n)$ (due to recursion stack).

Topic-2 :- Solution-1 —

#include <stdio.h>

#include <stdlib.h>

void swap (int* a, int* b) {

int t = *a;

*a = *b;

*b = t;

}

int partition (int arr[], int low, int high) {

int pivot = arr[high];

int i = (low-1);

for (int j = low; j <= high-1; j++) {

if (arr[j] <= pivot) {

i++;

swap(&arr[i], &arr[j]);

}

}

swap(&arr[i+1], &arr[high]);

return (i+1);

}

void quicksort (int arr[], int low, int high) {

if (low < high) {

int pi = partition(arr, low, high);

quicksort(arr, low, pi-1);

quicksort(arr, pi+1, high);

}

}

int main() {

int n;

if (scanf("%d", &n) != 1) {


```

    return 0;
}
int * timings = (int *) malloc (n * sizeof(int));
if (timings == null) {
    return 1;
}
for (int i=0; i<n; i++) {
    if (scanf ("%d", &timings[i]) != 1) {
        free(timings);
        return 0;
    }
}
quicksort (timings, 0, n-1);
for (int i=0; i<n; i++) {
    printf ("%d", timings[i]);
    if (i<n-1) {
        printf (" ");
    }
}
printf ("\n");
free (timings);
return 0;
}

```

main.c

```
1  #include <stdio.h>
2
3  // Function to swap two elements
4  void swap(int* a, int* b) {
5      int t = *a;
6      *a = *b;
7      *b = t;
8  }
9
10 // Partition function for Quick Sort
11 int partition(int arr[], int low, int high) {
12     int pivot = arr[high]; // Choosing the last element as pivot
13     int i = (low - 1); // Index of smaller element
14
15     for (int j = low; j <= high - 1; j++) {
16         // If current element is smaller than the pivot
17         if (arr[j] < pivot) {
18             i++;
19             swap(&arr[i], &arr[j]);
20         }
21     }
22     swap(&arr[i + 1], &arr[high]);
23     return (i + 1);
24 }
25
26 // Main Quick Sort function
27 void quickSort(int arr[], int low, int high) {
28     if (low < high) {
29         // Partition index
30         int pi = partition(arr, low, high);
31
32         // Separately sort elements before and after partition
33         quickSort(arr, low, pi - 1);
34         quickSort(arr, pi + 1, high);
35     }
36 }
```

```

20     },
21     }
22     swap(&arr[i + 1], &arr[high]);
23     return (i + 1);
24 }
25
26 // Main Quick Sort function
27 void quickSort(int arr[], int low, int high) {
28     if (low < high) {
29         // Partition index
30         int pi = partition(arr, low, high);
31
32         // Separately sort elements before and after pa
33         quickSort(arr, low, pi - 1);
34         quickSort(arr, pi + 1, high);
35     }
36 }
37
38 int main() {
39     int N;
40     scanf("%d", &N);
41
42     int timings[N];
43     for(int i = 0; i < N; i++) {
44         scanf("%d", &timings[i]);
45     }
46
47     quickSort(timings, 0, N - 1);
48
49     for(int i = 0; i < N; i++) {
50         printf("%d ", timings[i]);
51     }
52     printf("\n");
53
54     return 0;
55 }

```



```

26 // Main Quick Sort function
27 void quickSort(int arr[], int low, int high) {
28     if (low < high) {
29         // Partition index
30         int pi = partition(arr, low, high);
31
32         // Separately sort elements before and after partition
33         quickSort(arr, low, pi - 1);
34         quickSort(arr, pi + 1, high);
35     }
36 }
37
38 int main() {
39     int N;
40     scanf("%d", &N);
41
42     int timings[N];
43     for(int i = 0; i < N; i++) {
44         scanf("%d", &timings[i]);
45     }
46
47     quickSort(timings, 0, N - 1);

```



```

6 55 49 62 58 45 51
45 49 51 55 58 62

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

Solution - 2 :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void swap (int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
int partition (int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap (&arr[i], &arr[j]);
        }
    }
    swap (&arr[i + 1], &arr[high]);
    return (i + 1);
}
int randomized_partition (int arr[], int low, int high) {
    int random_index = low + rand() % (high - low + 1);
    swap (&arr[random_index], &arr[high]);
    return partition (arr, low, high);
}
void quicksort (int arr[], int low, int high) {
    if (low < high) {
        int pi = randomized_partition (arr, low, high);
        quicksort (arr, low, pi - 1);
    }
}
```

Date / /
Page No.

```
quicksort(arr, pi+1, high);  
}
```

```
int main() {
```

```
    srand(time(NULL));
```

```
    int n;
```

```
    if (scanf("%d", &n) != 1) return 0;
```

```
    int * book_ids = (int *) malloc(n * sizeof(int));
```

```
    if (book_ids == null) return 1;
```

```
    for (int i=0; i<n; i++) {
```

```
        if (scanf("%d", &book_ids[i]) != 1) return 0;
```

```
    }  
    quicksort(book_ids, 0, n-1);
```

```
    for (int i=0; i<n; i++) {
```

```
        printf("%d", book_ids[i]);
```

```
        if (i < n-1) {
```

```
            printf(" ");
```

```
        }
```

```
    }
```

```
    printf("\n");
```

```
    free(book_ids);
```

```
    return 0;
```

```
}
```




main.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void swap(long long* a, long long* b) {
5      long long t = *a;
6      *a = *b;
7      *b = t;
8  }
9
10 int partition(long long arr[], int low, int high) {
11     long long pivot = arr[high];
12     int i = (low - 1);
13
14     for (int j = low; j <= high - 1; j++) {
15         if (arr[j] < pivot) {
16             i++;
17             swap(&arr[i], &arr[j]);
18         }
19     }
20     swap(&arr[i + 1], &arr[high]);
21     return (i + 1);
22 }
23
24 void quickSort(long long arr[], int low, int high) {
25     if (low < high) {
26         int pi = partition(arr, low, high);
27         quickSort(arr, low, pi - 1);
28         quickSort(arr, pi + 1, high);
29     }
30 }
31
32 int main() {
33     int N;
34     scanf("%d", &N);
35
36     // Book IDs can be up to 10^9, using long long for safety
```

```

17         swap(&arr[i], &arr[j]);
18     }
19 }
20 swap(&arr[i + 1], &arr[high]);
21 return (i + 1);
22 }
23
24 void quickSort(long long arr[], int low, int high) {
25     if (low < high) {
26         int pi = partition(arr, low, high);
27         quickSort(arr, low, pi - 1);
28         quickSort(arr, pi + 1, high);
29     }
30 }
31
32 int main() {
33     int N;
34     scanf("%d", &N);
35
36     // Book IDs can be up to 10^9, using long long for safety
37     long long *bookIDs = (long long *)malloc(N * sizeof(long long));
38     for(int i = 0; i < N; i++) {
39         scanf("%lld", &bookIDs[i]);
40     }
41
42     quickSort(bookIDs, 0, N - 1);
43
44     for(int i = 0; i < N; i++) {
45         printf("%lld ", bookIDs[i]);
46     }
47     printf("\n");
48
49     free(bookIDs);
50     return 0;
51 }

```

```

23
24 void quickSort(long long arr[], int low, int high) {
25     if (low < high) {
26         int pi = partition(arr, low, high);
27         quickSort(arr, low, pi - 1);
28         quickSort(arr, pi + 1, high);
29     }
30 }
31
32 int main() {
33     int N;
34     scanf("%d", &N);
35
36     // Book IDs can be up to 10^9, using long long for safety
37     long long *bookIDs = (long long *)malloc(N * sizeof(long long));
38     for(int i = 0; i < N; i++) {
39         scanf("%lld", &bookIDs[i]);
40     }
41
42     quickSort(bookIDs, 0, N - 1);
43

```



input

```

5 1025 1001 1050 1010 1005
1001 1005 1010 1025 1050

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```


Solution - 3 :-

```
#include <stdio.h>
#include <stdlib.h>

void swap (int * a, int * b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition (int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low-1);
    for (int j = low; j < high; j++) {
        if (arr[j] >= pivot) {
            i++;
            swap (&arr[i], &arr[j]);
        }
    }
    swap (&arr[i+1], &arr[high]);
    return (i+1);
}

void quicksortDescending (int arr[], int low, int high) {
    if (low < high) {
        int pivot_index = partition (arr, low, high);
        quicksortDescending (arr, low, pivot_index - 1);
        quicksortDescending (arr, pivot_index + 1, high);
    }
}

int main() {
    int n;
    if (scanf ("%d", &n) != 1) return 0;
    int * sales = (int *) malloc (n * sizeof (int));
```

```

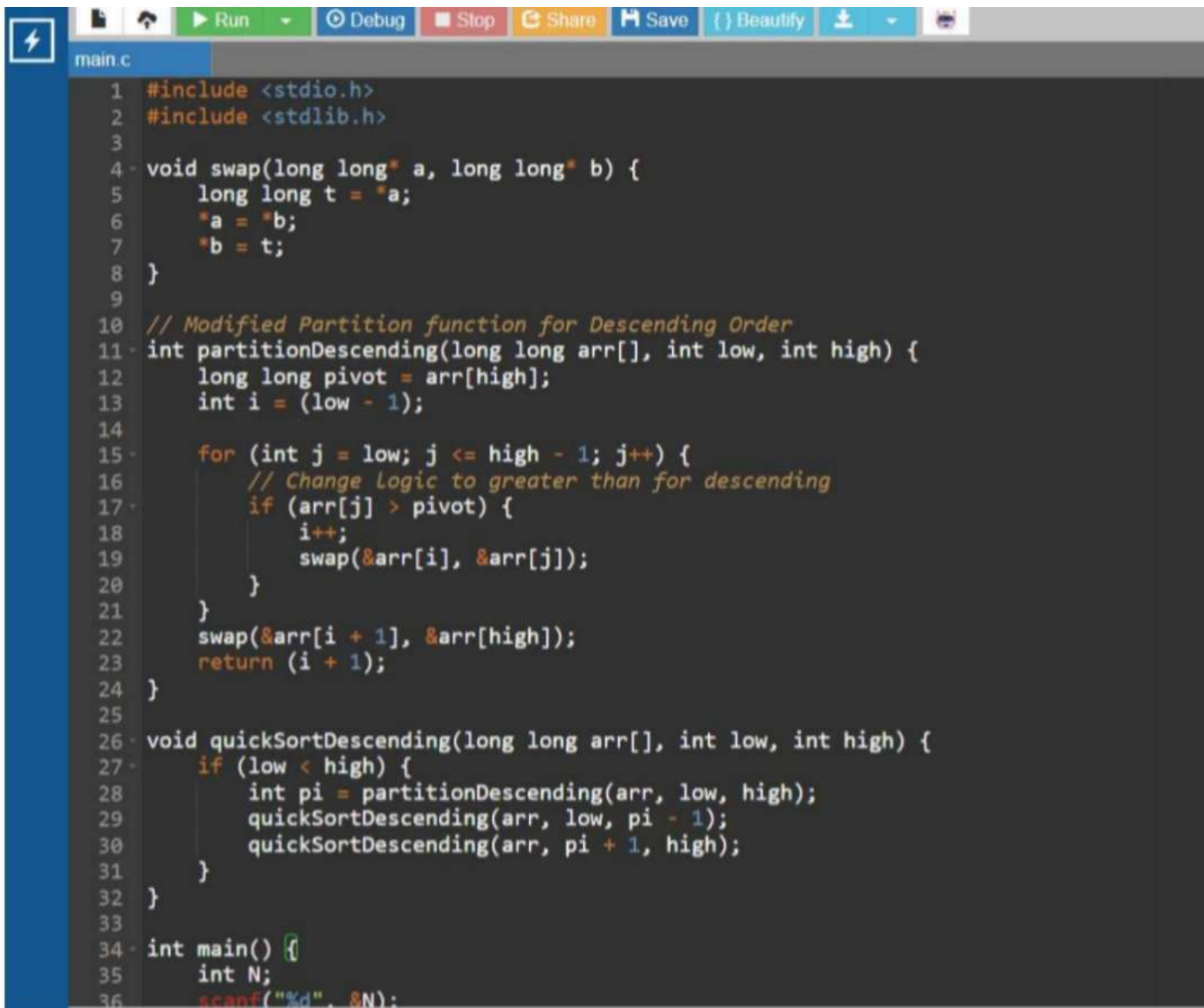
if (sales == Null) return 1;
for (int i=0; i<n; i++) {
    if (scanf("%d", &sales[i]) != 1) {
        free(sales);
        return 0;
    }
}

quicksortDescending(sales, 0, n-1);
for (int i=0; i<n; i++) {
    printf("%d", sales[i]);
    if (i < n-1) {
        printf(" ");
    }
}

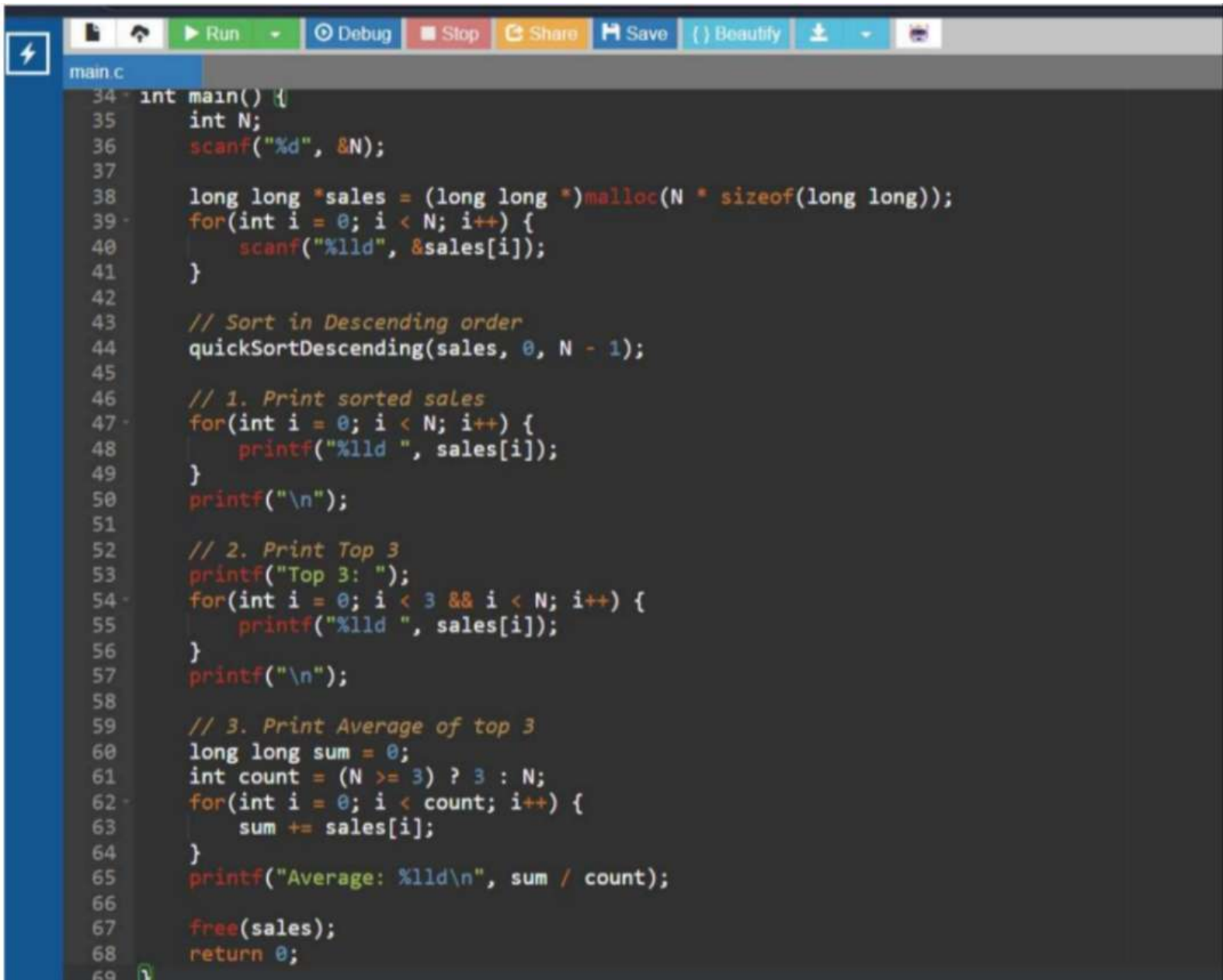
printf("\n");
printf("Top 3 : %d %d %d\n", sales[0],
                                           sales[1], sales[2]);
double sum = sales[0] + sales[1] + sales[2];
double avg = sum/3.0;
if ((long long)avg == avg) {
    printf("Average : %i of\n", avg);
} else {
    printf("Average : %.2f\n", avg);
}

free(sales);
return 0;
}

```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void swap(long long* a, long long* b) {
5     long long t = *a;
6     *a = *b;
7     *b = t;
8 }
9
10 // Modified Partition function for Descending Order
11 int partitionDescending(long long arr[], int low, int high) {
12     long long pivot = arr[high];
13     int i = (low - 1);
14
15     for (int j = low; j <= high - 1; j++) {
16         // Change Logic to greater than for descending
17         if (arr[j] > pivot) {
18             i++;
19             swap(&arr[i], &arr[j]);
20         }
21     }
22     swap(&arr[i + 1], &arr[high]);
23     return (i + 1);
24 }
25
26 void quickSortDescending(long long arr[], int low, int high) {
27     if (low < high) {
28         int pi = partitionDescending(arr, low, high);
29         quickSortDescending(arr, low, pi - 1);
30         quickSortDescending(arr, pi + 1, high);
31     }
32 }
33
34 int main() {
35     int N;
36     scanf("%d", &N);
```

The image shows a code editor window with a toolbar at the top containing icons for Run, Debug, Stop, Share, Save, Beautify, and a download icon. The file name 'main.c' is visible in the tab. The code is a C program that reads an integer N, allocates memory for an array of long long integers, reads N values, sorts them in descending order using quickSortDescending, and then prints the sorted array, the top 3 values, and the average of the top 3 values.

```
34 int main() {
35     int N;
36     scanf("%d", &N);
37
38     long long *sales = (long long *)malloc(N * sizeof(long long));
39     for(int i = 0; i < N; i++) {
40         scanf("%lld", &sales[i]);
41     }
42
43     // Sort in Descending order
44     quickSortDescending(sales, 0, N - 1);
45
46     // 1. Print sorted sales
47     for(int i = 0; i < N; i++) {
48         printf("%lld ", sales[i]);
49     }
50     printf("\n");
51
52     // 2. Print Top 3
53     printf("Top 3: ");
54     for(int i = 0; i < 3 && i < N; i++) {
55         printf("%lld ", sales[i]);
56     }
57     printf("\n");
58
59     // 3. Print Average of top 3
60     long long sum = 0;
61     int count = (N >= 3) ? 3 : N;
62     for(int i = 0; i < count; i++) {
63         sum += sales[i];
64     }
65     printf("Average: %lld\n", sum / count);
66
67     free(sales);
68     return 0;
69 }
```

```

40         scanf("%lld", &sales[i]);
41     }
42
43     // Sort in Descending order
44     quickSortDescending(sales, 0, N - 1);
45
46     // 1. Print sorted sales
47     for(int i = 0; i < N; i++) {
48         printf("%lld ", sales[i]);
49     }
50     printf("\n");
51
52     // 2. Print Top 3
53     printf("Top 3: ");
54     for(int i = 0; i < 3 && i < N; i++) {
55         printf("%lld ", sales[i]);
56     }
57     printf("\n");
58
59     // 3. Print Average of top 3
60     long long sum = 0;
61     int count = (N >= 3) ? 3 : N;

```



input

```

6 1200 5000 3000 2500 7000 4500
7000 5000 4500 3000 2500 1200
Top 3: 7000 5000 4500
Average: 5500

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```